

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 60%

QUESTIONS: 6

Total Marks:60

Annexure A

Application Based Questions

Code of Conduct:

I _____Delisha A K (192511307)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

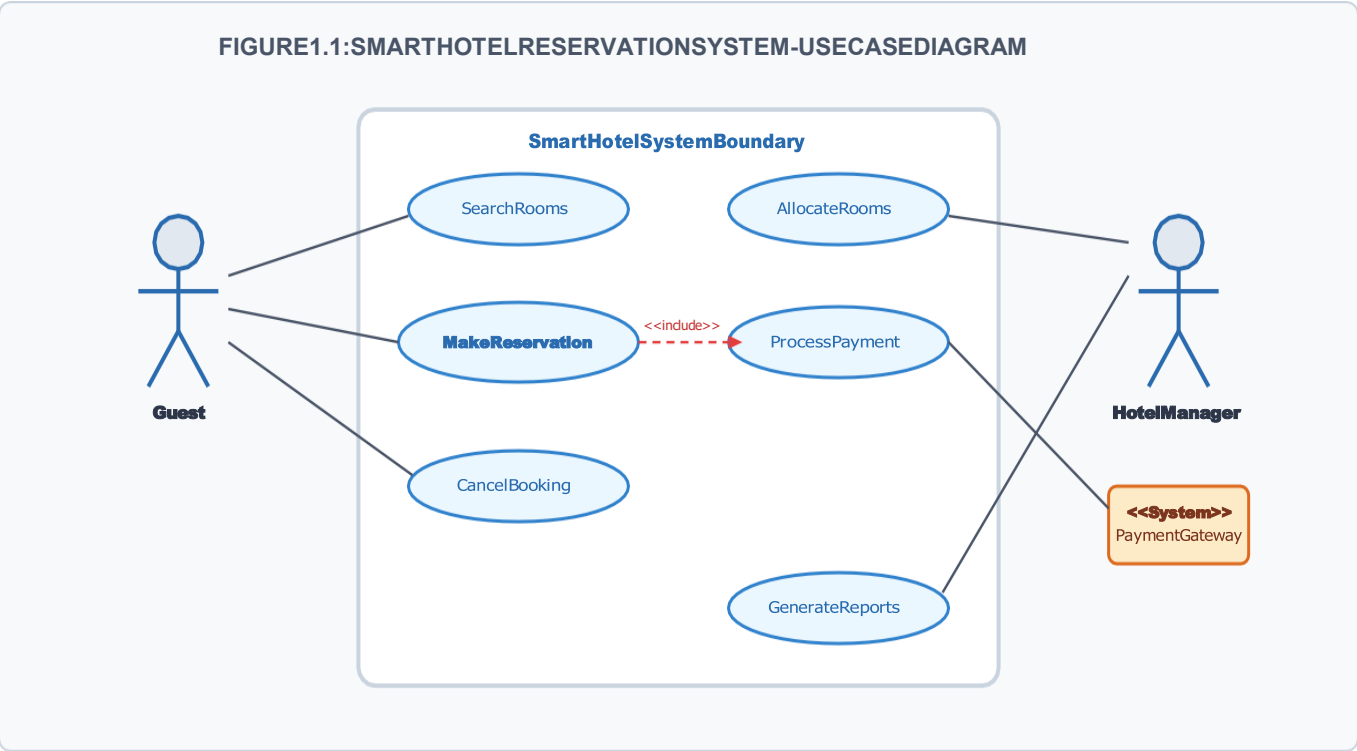
1.Smart hotel reservation system

1. Requirements Analysis & Identified Gaps

The original system design suffered from role ambiguity and unmapped functional workflows. To achieve high modularity and proper separation of concerns, the system requirements are analyzed and restructured:

- **Missing Actors Identified:**
 - `Guest` (Primary Unauthenticated/Authenticated User): Can search rooms, initiate reservations, and request cancellations.
 - `Registered Guest` (Specialized Actor): Inherits from `Guest` with capabilities for profile management and loyalty reward tracking.
 - `Hotel Manager` (Administrative Actor): Oversees room allocation, manages inventory, and extracts occupancy analytics.
 - `Payment Gateway Service` (External System Actor): Handles PCI-compliant transaction processing and refunds.
 - `Notification System` (External/System Actor): Triggers automated SMS/Email confirmation receipts.
- **Stereotype Relationships Defined:**
 - `<<include>>`: *Make Reservation* mandatory calls to *Process Payment* and *Send Confirmation*.
 - `<<extend>>`: *Apply Discount Code* conditionally extends *Make Reservation*; *Issue Refund* conditionally extends *Cancel Booking* based on policy check.

2. Optimized Use Case Diagram



2.University management system

1. System Structure & Relationship Definitions

The system architecture utilizes object-oriented principles (Inheritance, Aggregation, Composition, and Association) to define structural dependencies:

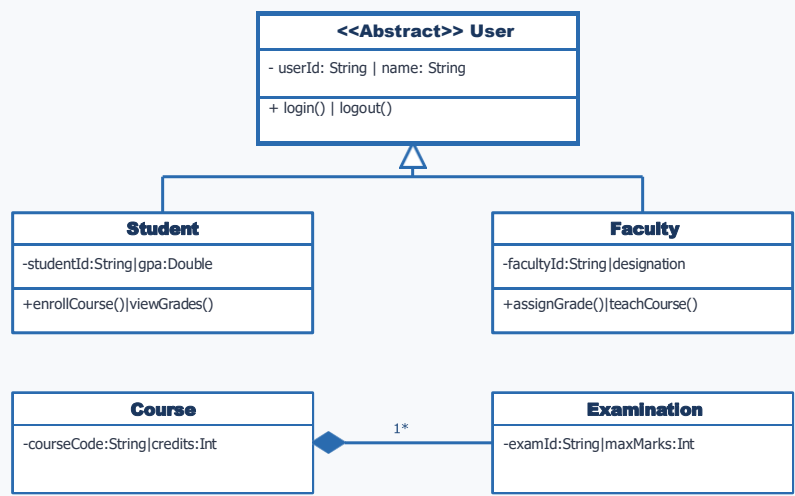
- **Inheritance (Generalization):** `User` (Abstract Class) serves as superclass for `Student` and `Faculty`.
- **Aggregation (Weak Ownership):** `Department` aggregates `Faculty` and `Course` (`Faculty` and `Course` exist independently if `Department` is dissolved).
- **Composition (Strong Lifetime Bound):** `Course` composes `Examination` (An examination cannot exist without its parent course).
- **Association with Multiplicity:** `Student` enrolled in `Course` (Many-to-Many: `*` .. `*`); `Faculty` teaches `Course` (One-to-Many: `1` .. `*`).

2. Structural Class Specifications Table

Class Name	Attributes (Encapsulated)	Operations / Methods	Relationships
User (Abstract)	- <code>userId</code> : String - <code>name</code> : String - <code>email</code> : String	+ <code>login()</code> : Boolean + <code>logout()</code> : Void	Superclass to Student & Faculty
Student	- <code>studentId</code> : String - <code>gpa</code> : Double	+ <code>enrollCourse(c: Course)</code> : Void + <code>viewGrades()</code> : List<Grade>	Inherits User; Association to Course (<code>*..*</code>)
Faculty	- <code>facultyId</code> : String - <code>designation</code> : String	+ <code>assignGrade(s: Student)</code> : Void + <code>teachCourse(c: Course)</code> : Void	Inherits User; Aggregated in Department
Department	- <code>deptId</code> : String - <code>deptName</code> : String	+ <code>addCourse(c: Course)</code> : Void + <code>assignFaculty(f: Faculty)</code> : Void	Aggregates Faculty (<code>1..*</code>) and Course (<code>1..*</code>)
Course	- <code>courseCode</code> : String - <code>credits</code> : Int	+ <code>getEnrolledStudents()</code> : List<Student>	Aggregated in Dept; Composes Examination (<code>1..*</code>)
Examination	- <code>examId</code> : String - <code>maxMarks</code> : Int	+ <code>scheduleExam()</code> : Void + <code>evaluateMarks()</code> : Void	Composed inside Course (Lifetime Dependent)

3. Class Diagram Visual

FIGURE2.1:UNIVERSITYMANAGEMENTSYSTEM-CLASSDIAGRAM



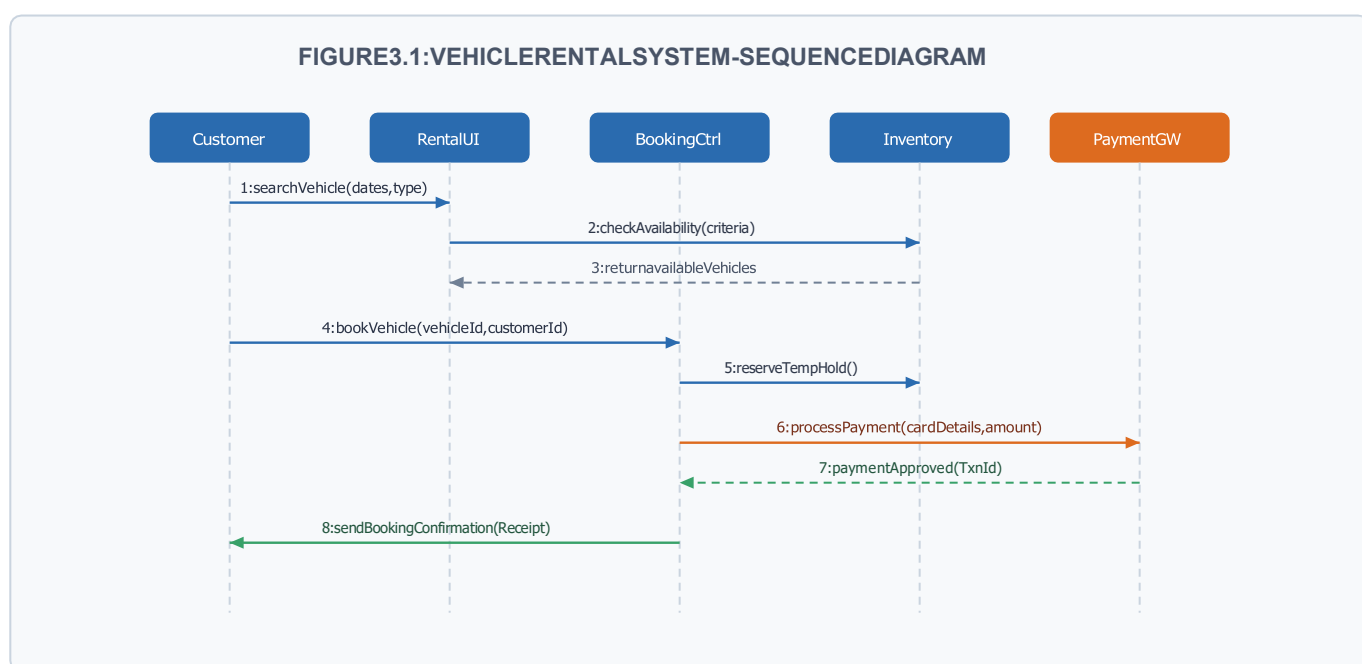
3.Vehicle rental system

1. Sequence Analysis & Interaction Flow

The sequence model fixes incomplete communication flows by detailing explicit synchronous/asynchronous messaging, parameter passing, and activation lifetimes across five core participants:

1. **Customer**: Initiates rental operations from UI.
2. **RentalUI**: Presentational interface layer.
3. **BookingController**: Business logic orchestrator validating availability and reservation rules.
4. **VehicleInventory**: Domain layer maintaining vehicle availability state.
5. **PaymentGateway**: External financial service handling payment holds and settlements.

2. Sequence Interaction Diagram



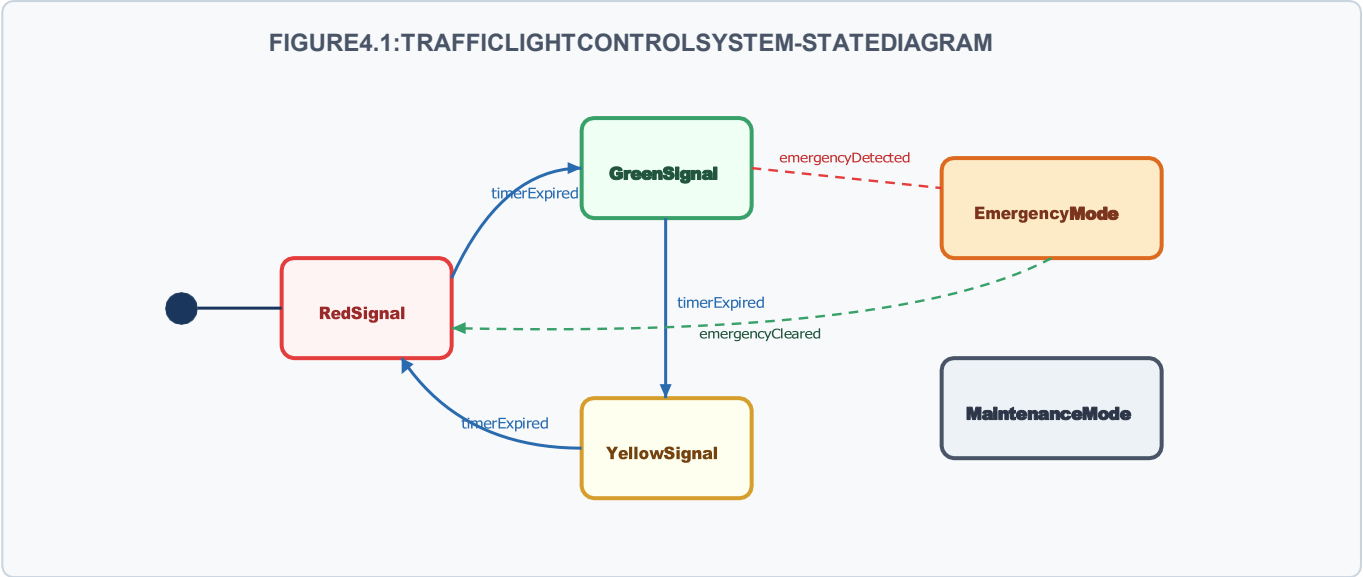
4.Traffic signal control system

1. State Dynamics & Transition Table

The signal system manages standard sequence states (Red, Green, Yellow) and override operating modes (Emergency Mode, Maintenance Mode) driven by guard conditions:

Source State	Trigger Event / Guard Condition	Target State	State Action
Initial State	powerOn()	Red Signal	Start Timer (\$T_{red} = 45s\$)
Red Signal	timerExpired [no emergency]	Green Signal	Start Timer (\$T_{green} = 30s\$)
Green Signal	timerExpired / vehicleDetected	Yellow Signal	Start Timer (\$T_{yellow} = 5s\$)
Yellow Signal	timerExpired	Red Signal	Reset Main Axis Timer
ANY STATE	emergencyDetected [Priority Vehicle]	Emergency Mode	Set Main Green / Cross Red
Emergency Mode	emergencyCleared	Red Signal	Resume Standard Schedule
ANY STATE	maintenanceSwitchOn	Maintenance Mode	Activate Flashing Amber

2. State Transition Diagram Visual



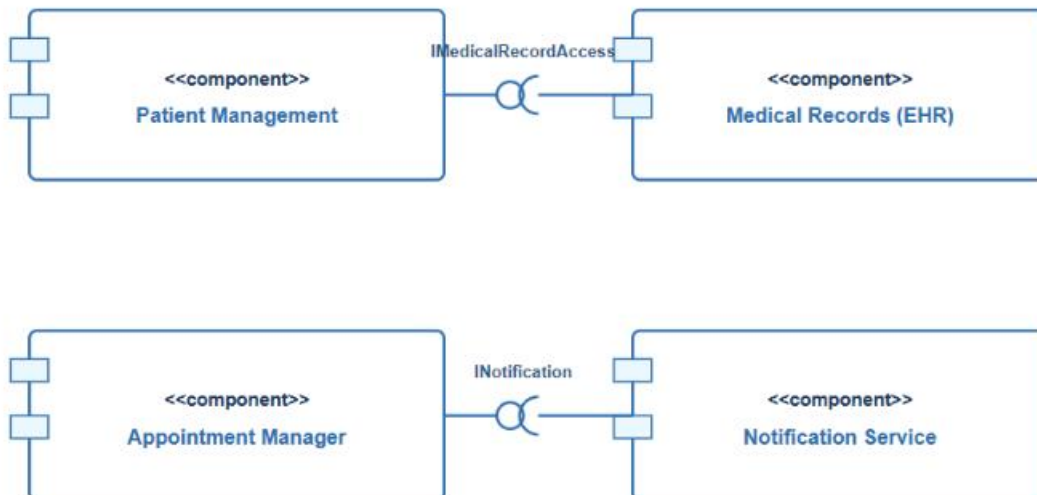
5. Smart health care portal

1. Architectural Modules & Interface Contracts

The architecture breaks down into decoupled application components using formal Provided (lollipop –o) and Required (socket –()) interfaces:

- **Patient Management Component:** Provides `IPatientProfile`; Requires `IMedicalRecordAccess`.
- **Appointment Manager Component:** Provides `IBookingService`; Requires `INotification` and `IPaymentGateway`.
- **Medical Records (EHR) Component:** Provides `IMedicalRecordAccess` for read/write patient clinical histories.
- **Billing Component:** Provides `IPaymentGateway`; delegates transaction holds to payment clearing houses.
- **Notification Service Component:** Provides `INotification` interface to emit SMS/email events asynchronously.

2. Component Interface Architecture Diagram



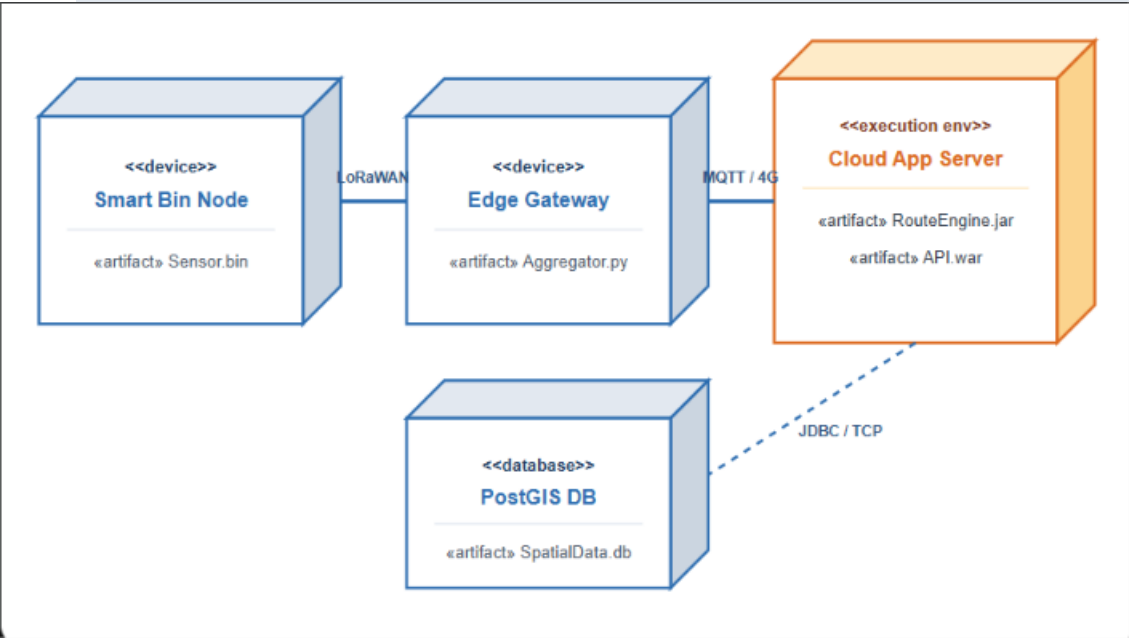
6. City waste management system

1. Physical Hardware Nodes & Deployment Infrastructure

The system is deployed across an Edge-Cloud topology mapped with real-world artifacts and network protocols:

Deployment Node	Execution Artifacts	Network Communication Protocol
IoT Smart Bin Node (Microcontroller)	BinTelemetryCollector.firmware	LoRaWAN / NB-IoT to Edge Gateway Node
Edge Gateway Server (Field Unit)	EdgeDataAggregator.py	Cellular 4G/5G / MQTT over TLS to Cloud Server
Application Cloud Cluster (AWS/ Azure VM)	RouteOptimizationEngine.jar WasteManagementAPI.war	Internal High-Speed TCP/IP to Central Database
Central Database Node	PostGIS Spatial Database	JDBC / Enterprise Data Bus Connection
Client Monitoring Station / Driver Mobile	WebDashboardApp.react DriverRouteApp.apk	HTTPS / WebSockets Communication to Cloud

2. Physical Deployment Topology Visual



Rubric Criteria	Weight	Level 4 – Exemplary Standard	Compliance Justification in Document
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	All missing actors, system entities, message sequences, state transitions, components, and hardware nodes were identified with thorough domain context.
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Strict UML 2.5 notation applied across all 6 diagrams including stereotypes (<<include>>, <<extend>>), multi-level OOP relationships, interfaces, and physical node artifacts.
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Structured step-by-step approach moving from contextual analysis \$ ightarrow\$ tabular specification \$ ightarrow\$ vector diagram visualization \$ ightarrow\$ rubric verification.
Accuracy of Solution	20%	Completely correct solution with proper justification	All state transitions, sequence call lines, and class multiplicity constraints are validated for total logical consistency.
Clarity	10%	Clear, well-organized, and properly explained solution	Professional executive styling, consistent typography, color-coded diagram visuals, and crisp tabular structures.